Changing the graphics engine on Windows

Airline Manager Pro · planning notes · 2 November 2026

Short answer

Yes. The graphics engine can be changed without destroying the game. How big the job is depends on which engine it is changed to.

How the game is built today

- The game is written in TypeScript. On the PC it runs in a Microsoft Edge app window; on Android in the system WebView.
- The 3D is drawn with **three.js** (MIT licence) through WebGL.
- About **three quarters of the code never touches the graphics engine**: the airline, money, flights, schedules, crews, the court, the simulator link (SimConnect), saves, menus and the map.
- The 3D drawing sits in about **19 files**: the 3D flight view, the cabin, the offices, the courtroom, the people (`src/graphics/human.ts`) and the airports. Only these would change.
- **Saves and player data are never at risk**: the engine only draws what the data says.
- All 3D is built in code (aircraft, people, offices, airports from published dimensions), not hand-made model files. This ports well: the same shapes are rewritten in the new engine's language rather than modelled again by hand.

Options, from easiest to hardest

Option	What changes	Rough time (human team)	Notes
three.js WebGPU renderer	Small changes to the renderer and some materials	Days to a couple of weeks	Better performance and effects; Android can keep WebGL; PC and Android keep sharing code.
Another web engine (Babylon.js, PlayCanvas)	The 3D files are rewritten	Weeks	Game, menus, data and the Android version stay as they are.
Full game engine for PC only (Unity, Unreal, Godot)	Every 3D scene rebuilt, plus a bridge so the engine and the game logic talk to each other	Months	PC and Android no longer share their 3D code. Check licences for commercial sale (see below).

How fast in our way of working

"Months" is what a small human team would need. With the code written in our working sessions, the code itself is more like weeks. What sets the pace is how often a version can be built, looked at and corrected:

- **The look needs rounds.** Every scene is built, run and checked from screenshots, then adjusted: lighting, materials, people and the cabin each take a few rounds to match today's quality.

- **Testing on your PC.** The simulator link and the gamepad only work on a real Windows PC with the simulator.
- **Where the engine runs.** Today everything is built and screenshotted in the working environment. Unity and Unreal are large, Windows-first editors, so builds would go through GitHub's Windows build machines: slower per round, but it works.

Engine	Fit for this game	Rough time (working sessions)	Licence
Godot	Fastest: small, runs in the working environment, builds Windows versions from there	About 2–4 weeks	MIT: free, no royalties
Unity	Very doable: can be driven entirely from C# code; built on GitHub	About 3–6 weeks	Unity Personal free up to a revenue limit; paid plans above it. Check current terms.
Unreal	Hardest: very large engine, slow C++ builds, GitHub's standard machines struggle	Longest	Royalty on sales above a threshold. Check current terms. Its strength (photoreal hand-made assets) isn't how this game is made.

These are rough estimates; the main uncertainty is how many rounds the visuals need to look right.

Recommendation

1. Prefer **Godot** or **Unity** over Unreal for this game.
2. Lower the risk: **port one scene first** (for example the cabin or the operations office), compare it side by side with today's version, then decide on the whole switch.
3. Until then, keep new 3D work behind the game's own small helpers (the people builder `Human.build`, the glow `AimGlow` and similar) rather than spreading three.js calls further, so a later switch has fewer, smaller pieces to rewrite.

This is planning information. Engine licences and prices change; check each engine's current terms before deciding.